CURRENT PROJECT AUDIT | 29 OCTOBER 2026

SkillSync Database Structure Reference

What the current frontend reads, writes, and expects from Supabase - with unknown database details clearly marked.

At a glance	Finding
Database client	Supabase JavaScript client used directly in the browser.
Application tables touched	profiles, user_roles, skills, profile_skills, posts, attachments.
Custom API server	No application API routes or controllers found in this project.
Schema verification	Base DDL and live Supabase connection were unavailable during this audit.

Important scope note

This document reports the current repository evidence, not a confirmed dump of the live database. No field type, nullability, default, primary key, or foreign key is asserted unless the checked-in code or SQL explicitly establishes it. The README refers to a database manual PDF, but that file was not available in the searched workspace.

Use the field lists as a frontend contract checklist. Before changing backend-dependent behavior, compare them with the live Supabase schema or the original database manual.

1. Tables and field contracts

The tables below are the application tables explicitly accessed by source code. “Unknown” means the base schema is not included in this repository; it does not mean the database column is nullable or optional.

public.profiles

Field	Frontend use / evidence	Type, required, default, keys
user_id	Profile row key/filter; associates author profiles with posts.	Unknown
full_name	Author display name; profile editor updates it.	Unknown
institution	Author institution display; profile editor updates it.	Unknown
bio	Profile editor updates it.	Unknown

Reads: user_id, full_name, institution. Writes: full_name, institution, bio, filtered by user_id.

public.user_roles

Field	Frontend use / evidence	Type, required, default, keys
user_id	Loads role rows for current user; role insert/delete ownership.	Unknown
role	Learner/instructor capability and post permission selection.	Unknown; app values learner or instructor.

Reads: role. Writes: user_id and role. Role migration policies restrict self-service inserts/deletes to own account and those two role values.

public.skills

Field	Frontend use / evidence	Type, required, default, keys
skill_id	Catalogue identifier and profile-skill association.	Unknown
skill_name	Skill label; duplicate detection and insert validation.	Unknown; policy checks trimmed length 2..60.
category	Catalogue label and embedded profile-skill data; new custom skills use Other.	Unknown

Reads: skill_id, skill_name, category. Writes: skill_name and category. Catalogue sorted by skill_name.

public.profile_skills

Field	Frontend use / evidence	Type, required, default, keys
user_id	Selects and mutates the signed-in user’s profile selections.	Unknown
skill_id	Selects a catalogue entry.	Unknown
skill_type	Learner/offering grouping.	Unknown; app values learning or offering.

Reads skill_id, skill_type and related skills(skill_name, category). Writes all three fields on insert; filters all three on delete.

public.posts

Field	Frontend use / evidence	Type, required, default, keys
post_id	Post identity for updates and RPC calls.	Unknown
user_id	Author identity; “My posts” filter.	Unknown
posting_role	Shown on the card; chosen at insert.	Unknown; app values learner or instructor.
description	Post text; course announcement title is encoded in this text.	Unknown
status	Published feed filter and draft/publish/archive UI.	Unknown; UI uses draft, published, archived.
created_at	Newest-first ordering and timestamp displayed to users.	Unknown

Query uses select(*) so every actual database column is returned. The UI explicitly reads the listed fields. Insert writes user_id, posting_role, description; draft edits update description.

public.attachments

Field	Frontend use / evidence	Type, required, default, keys
attachment_id	Attachment identity for unregister/removal.	Unknown
post_id	Groups attachment rows under posts.	Unknown
original_name	Filename shown in the feed.	Unknown
byte_size	Displayed file size.	Unknown
bucket_id	Storage bucket selection, with client fallback.	Unknown
object_path	Storage object download/removal path.	Unknown

Query uses select(*) so all actual columns are returned. The UI reads the listed fields. Metadata registration/removal uses RPCs.

2. Relationships and managed services

These are the relationships established by query patterns or migration behavior. Where a real foreign-key definition is not included, the relationship is described as an application association rather than a confirmed constraint.

From	To / behavior	Evidence level
posts.user_id	profiles.user_id for author lookup	Query association; FK not verified.
profile_skills.skill_id	skills.skill_id via skills(skill_name,category)	Supabase embedded relation works in expected schema; exact FK/key definition unavailable.
user_roles.user_id, role	Compared with posts.user_id, posting_role by trigger	Trigger logic verified; FK/unique constraints unknown.
attachments.post_id	Post rows	App groups attachment rows by post_id; FK not verified.
App user.id	user_id fields in app queries/writes	Auth ID is used as identifier; underlying SQL FK not verified.

Supabase Auth and Storage

Auth is accessed through getSession, signUp, signInWithPassword, onAuthStateChange, signOut, and updateUser. The app reads user.id, email, and user_metadata. Signup submits full_name, institution, role_choice metadata. Profile editing updates Auth metadata full_name, institution, university, program, and bio.

Storage uses the private bucket post-attachments. The checked-in SQL sets the bucket to private, limits each object to 10,485,760 bytes, and allows image/jpeg, image/png, image/webp, application/pdf, and video/mp4. The migration modifies storage.buckets; it does not document its full schema.

RPC functions called by the client

RPC	Client arguments	Returned value usage
register_post_attachment	p_post_id, p_object_path, p_original_name	Only error checked.
set_post_status	p_post_id, p_target_status	Only error checked; target used as published or archived.
unregister_post_attachment	p_attachment_id	Returned bucket_id and object_path are used, with fallbacks.
delete_draft_post	p_post_id	Only error checked.

The SQL definitions/signatures/return types of these RPC functions are not present, so no additional contract is inferred.

3. Frontend data returned by Supabase

Operation	Requested fields	What the code consumes
Skill catalogue	skills: skill_id, skill_name, category	Catalogue rows and custom-skill insert response.
Profile skills	profile_skills: skill_id, skill_type, skills(skill_name,category)	Mapped objects: skill_id, skill_type, skill_name, category.
Role list	user_roles: role	String role values.
Posts	posts: *	Every actual post column; component reads post_id, user_id, posting_role, description, status, created_at.
Attachments	attachments: *	Every actual attachment column; component reads attachment_id, post_id, original_name, byte_size, bucket_id, object_path.
Author profiles	profiles: user_id, full_name, institution	Those three fields only.
Profile update	Update full_name, institution, bio by user_id	No profile response row is selected or consumed; Auth updateUser returns the updated user.
Role deletion	Delete by user_id and role, select role	Deleted role row (maybeSingle); insert response body unused.

Stored-only fields cannot be enumerated from this repository. The posts and attachments queries select every column, but their complete schemas are unknown. For skills, profile skills, roles, and author profiles, the code selects only the fields shown above; other stored columns, if any, are not requested by those queries.

No custom HTTP API

No fetch/axios routes, server handlers, or controllers were found. The API boundary is Supabase Auth, PostgREST table operations, Storage operations, and the four RPC names listed above.

4. Example response shapes

Synthetic examples below show the shape consumed by the frontend. They do not confirm SQL types or the actual ID format. The posts and attachment list queries return additional fields if those columns exist because they use select(*).

Skill-data loader (composed client result)

```
{
  "catalog": [{
    "skill_id": "<database skill ID>",
    "skill_name": "Python",
    "category": "Programming"
  }],
  "profileSkills": [{
    "skill_id": "<database skill ID>",
    "skill_type": "learning",
    "skill_name": "Python",
    "category": "Programming"
  }],
  "roles": ["learner", "instructor"]
}
```

Post row (fields read by the UI)

```
{
  "post_id": "<database post ID>",
  "user_id": "<Supabase Auth user ID>",
  "posting_role": "instructor",
  "description": "Course: Python Basics\n\nLearn Python fundamentals.",
  "status": "published",
  "created_at": "<database timestamp>"
}
```

Attachment row (fields read by the UI)

```
{
  "attachment_id": "<database attachment ID>",
  "post_id": "<database post ID>",
  "original_name": "lesson-notes.pdf",
  "byte_size": 183204,
  "bucket_id": "post-attachments",
  "object_path": "<stored object path>"
}
```

Author profile lookup row

```
{
  "user_id": "<Supabase Auth user ID>",
  "full_name": "Ayesha Rahman",
  "institution": "Example University"
}
```

Role list row

```
{ "role": "learner" }
```

5. Frontend expectations to verify

No confirmed frontend/database mismatch can be proven without the live schema. Before changing backend-dependent UI, verify these expectations against Supabase:

• profiles supports full_name, institution, bio updates keyed by user_id; author reads expose user_id, full_name, institution.

• skills exposes skill_id, skill_name, category and accepts inserts for skill_name/category.

• profile_skills accepts user_id, skill_id, skill_type and exposes the skills relation used by PostgREST.

• user_roles accepts user_id and role values learner/instructor. The signup flow puts role_choice in Auth metadata, but this frontend does not insert user_roles during signup; role provisioning depends on backend behavior not present here.

• posts exposes post_id, user_id, posting_role, description, status, created_at and supports the RPC workflows.

• attachments exposes the fields consumed by the component and the unregister RPC returns bucket_id/object_path or equivalent data.

• program is read from and saved to Auth user_metadata; it is not part of the profiles table update in this frontend.

• Classroom/course/request data is currently prototype/demo data in the frontend; do not treat it as a confirmed database schema.

Repository evidence

Primary files inspected: src/lib/supabase.js, src/lib/skillService.js, src/components/DiscoverPosts.jsx, src/App.jsx, src/pages/ProfilePage.jsx, and the checked-in SQL files under supabase/. The SQL files are policy/configuration migrations, not base table DDL.

Source of truth limitation: the project README names Skill_Sharing_Database_Developer_Manual-1.pdf as the schema reference, but that PDF and live Supabase credentials were unavailable in this workspace during this audit.